“Pick Up Restaurante Piplup”

Manual Técnico

CDMX 14 de Abril de 2025

Contenido

1. Introducción 3
2. Descripción General del Sistema 3
3. Arquitectura del Sistema 4
4. Estructura del Proyecto 4
5. Tecnologías Utilizadas 5
6. Requisitos del Sistema 6
7. Instalación y Configuración 6
8. Especificación Técnica 8
9. Funcionalidades Clave 8
10. Descripción del Código 9
11. Flujo de Datos 16
12. Análisis de Casos de Uso 17
13. Diseño de la Aplicación 26
14. Validaciones y Manejo de Errores 26
15. Seguridad 27
16. Pruebas y Validación 28
17. Conclusión 29
18. Fuentes 30

1. Introducción

Pick Up es una aplicación móvil desarrollada en Kotlin para Android que permite a los usuarios realizar pedidos de comida de forma digital desde dispositivos móviles, facilitando la modalidad de recolección en ventanilla. Este manual proporciona la documentación técnica completa de la aplicación, desarrollada en equipo integrando conocimientos y habilidades de todos los integrantes.

La finalidad de este documento es servir como una guía completa que detalle tanto el diseño como la implementación, uso y mantenimiento del sistema, permitiendo la correcta gestión y evolución de este a futuro. Está dirigido a desarrolladores, usuarios técnicos y personal de soporte.

2. Descripción General del Sistema

La aplicación Pick Up permite realizar pedidos de comida de forma digital desde dispositivos móviles, facilitando la modalidad de recolección en ventanilla. Su diseño está centrado en el usuario, ofreciendo una navegación simple, eficiente y segura.

El sistema se compone de un cliente móvil desarrollado en Kotlin y un backend en Firebase, lo que garantiza rendimiento, disponibilidad y escalabilidad. Incluye procesos de autenticación, gestión de productos, carrito de compras, pagos electrónicos y registro de pedidos con seguimiento.

Características Principales:

- Interfaz de usuario intuitiva y moderna
- Gestión de carrito de compras en tiempo real
- Autenticación segura con Firebase
- Sincronización offline para mejor experiencia
- Historial de pedidos completo
- Múltiples métodos de pago

3. Arquitectura del Sistema

La aplicación emplea una arquitectura **MVVM (Model-View-ViewModel)** con enfoque **offline-first**, garantizando funcionamiento incluso sin conexión a internet.

Componentes Principales:

- **Frontend:** Kotlin con Jetpack Compose y Material Design 3
- **Backend:** Firebase (Authentication, Firestore)
- **Base de Datos Local:** Room Database
- **Sincronización:** WorkManager para tareas en segundo plano
- **Navegación:** Navigation Component

Diagrama de Arquitectura MVVM:

View (Activity/Fragment) → ViewModel → Repository → (Firestore/Room)

4. Estructura del Proyecto

```
com.example.pickup
├── activities/        # Actividades principales
├── adapters/          # Adaptadores para RecyclerView
├── data/              # Modelos y acceso a datos
│   ├── local/         # Room Database
│   └── remote/        # Firebase
├── models/            # Modelos de datos
├── utils/             # Utilidades y helpers
├── workers/           # Tareas en segundo plano (WorkManager)
├── viewmodels/        # ViewModels para MVVM
└── repositories/      # Repositorios para acceso a datos
```

Archivos Clave:

- MainActivity.kt - Actividad principal
- AppDatabase.kt - Configuración de Room
- FirebaseManager.kt - Gestión de Firebase
- CarritoManager.kt - Gestión del carrito (Singleton)
- SyncWorker.kt - Sincronización offline

5. Tecnologías Utilizadas

Dependencias Principales (build.gradle):

```
dependencies {
    // AndroidX y Material Design
    implementation 'androidx.core:core-ktx:1.9.0'
    implementation 'androidx.appcompat:appcompat:1.6.1'
    implementation 'com.google.android.material:material:1.9.0'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'

    // Firebase
    implementation 'com.google.firebase:firebase-auth-ktx:22.1.2'
    implementation 'com.google.firebase:firebase-firestore-ktx:24.8.1'

    // Room Database
    implementation 'androidx.room:room-runtime:2.5.2'
    kapt 'androidx.room:room-compiler:2.5.2'
    implementation 'androidx.room:room-ktx:2.5.2'

    // Navigation Component
    implementation 'androidx.navigation:navigation-fragment-ktx:2.6.0'
    implementation 'androidx.navigation:navigation-ui-ktx:2.6.0'

    // WorkManager
    implementation 'androidx.work:work-runtime-ktx:2.8.1'



    // Lifecycle
    implementation 'androidx.lifecycle:lifecycle-viewmodel-ktx:2.6.2'
    implementation 'androidx.lifecycle:lifecycle-livedata-ktx:2.6.2'
```

```
    // Glide (para imágenes)
    implementation 'com.github.bumptech.glide:glide:4.15.1'
    kapt 'com.github.bumptech.glide:compiler:4.15.1'

    // Coroutines
    implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-android:1.6.4'
}
```

6. Requisitos del Sistema

Para garantizar el correcto funcionamiento de la aplicación, se requieren los siguientes recursos:

Requisitos del Usuario:

- Dispositivo Android con versión 7.0 (API 24) o superior
- Conectividad a Internet (mínimo 2 Mbps recomendado)
- Permisos de almacenamiento, cámara y notificaciones activados
- Espacio libre en memoria de al menos 100 MB

Requisitos de Desarrollo:

- Android Studio Arctic Fox o superior
- Cuenta Firebase con proyecto configurado
- JDK 11 o superior
- Gradle 7.0+
- Navegador web actualizado

7. Instalación y Configuración

7.1 Instalación por APK

1. Descargar el archivo APK desde el repositorio oficial
2. Activar 'Fuentes desconocidas' desde Configuración > Seguridad
3. Ejecutar el archivo e instalar
4. Otorgar los permisos solicitado.

7.2 Configuración de Firebase

1. Crear proyecto en Firebase Console
2. Descargar google-services.json y colocarlo en app/
3. Habilitar Authentication (Email/Password) y Firestore
4. Configurar reglas de seguridad en Firestore:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
    match /comidas/{document} {
      allow read: if request.auth != null;
    }
    match /historial/{document} {
      allow read, write: if request.auth != null;
    }
  }
}
```

7.3 Configuración de Room Database

Las entidades están definidas en AppDatabase.kt. Las migraciones se manejan automáticamente con @Database(version = N).

8. Especificación Técnica

8.1 Arquitectura Detallada

El sistema se comunica mediante eventos asincrónicos y operaciones CRUD sobre colecciones como 'usuarios', 'productos', 'pedidos' y 'pagos'. La lógica se distribuye en capas: presentación, lógica de negocio y acceso a datos, con uso de ViewModels y coroutines para mantener la responsividad.

8.2 Gestión de Estados

- **UI State:** Manejo con StateFlow y LiveData
- **Navigation State:** Navigation Component con Safe Args
- **Data State:** Repository Pattern con Single Source of Truth

8.3 Manejo de Concurrencia

Uso extensivo de Kotlin Coroutines para operaciones asíncronas:

- Dispatchers.IO para operaciones de red y base de datos
- Dispatchers.Main para actualizaciones de UI
- Dispatchers.Default para cálculos pesados

9. Funcionalidades Clave

9.1 Autenticación

- **LoginActivity:** Maneja el inicio de sesión con Firebase Auth
- **Registro:** Registro de nuevos usuarios con validación de campos
- **Recuperación de contraseña:** Envío de email para restablecer contraseña

9.2 Menú y Carrito

- **MenuActivity:** Muestra productos filtrados por categoría
- **CarritoActivity:** Gestiona productos seleccionados antes del pago
- **CarritoManager:** Clase singleton que administra el carrito en memoria

9.3 Proceso de Pago

- **PagoActivity:** Muestra resumen del pedido y opciones de pago
- **PaymentSettingsActivity:** Configuración de métodos de pago (tarjeta/PayPal)
- **Validación de pagos:** Verificación de datos de tarjeta y procesamiento seguro

9.4 Historial de Pedidos

- **HistorialActivity:** Lista de pedidos anteriores con detalles
- **Sincronización Offline:** WorkManager actualiza datos cuando hay conexión
- **Filtros:** Por fecha, estado y tipo de pedido

9.5 Perfil de Usuario

- **PerfilActivity:** Muestra información del usuario y permite editar datos
- **Configuraciones:** Preferencias de notificaciones y privacidad

10. Descripción del Código

```
package com.example.pickup

import android.animation.AnimatorSet
import android.animation.ObjectAnimator
import android.content.Intent
import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.view.View
import android.view.animation.AccelerateDecelerateInterpolator
import androidx.appcompat.app.AppCompatActivity
import com.google.firebase.FirebaseApp
import com.google.firebase.auth.FirebaseAuth

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        FirebaseApp.initializeApp(this)
        setContentView(R.layout.activity_main)
```

```
// Referencias a vistas
val logoImageView = findViewById<View>(R.id.logoImageView)
val bienvenidoText = findViewById<View>(R.id.bienvenidoText)
val dot1 = findViewById<View>(R.id.dot1)
val dot2 = findViewById<View>(R.id.dot2)
val dot3 = findViewById<View>(R.id.dot3)

// Animación de logo y texto (fadeIn + slideUp)
val logoAnimator = AnimatorSet().apply {
    playTogether(
        ObjectAnimator.ofFloat(logoImageView, "alpha", 0f, 1f),
        ObjectAnimator.ofFloat(logoImageView, "translationY", 40f, 0f)
    )
    duration = 1000
    interpolator = AccelerateDecelerateInterpolator()
}

val textAnimator = AnimatorSet().apply {
    playTogether(
        ObjectAnimator.ofFloat(bienvenidoText, "alpha", 0f, 1f),
        ObjectAnimator.ofFloat(bienvenidoText, "translationY", 40f, 0f)
    )
    duration = 1000
    interpolator = AccelerateDecelerateInterpolator()
    startDelay = 500
}

val introSet = AnimatorSet()
introSet.playSequentially(logoAnimator, textAnimator)
```

```
introSet.start()

// Función para crear animaciones de escala para los puntitos
fun createScaleAnimator(view: View, delay: Long) = ObjectAnimator.ofFloat(view, "scaleX", 1f, 1.6f, 1f).apply {
    duration = 600
    startDelay = delay
    repeatCount = ObjectAnimator.INFINITE
    repeatMode = ObjectAnimator.RESTART
    interpolator = AccelerateDecelerateInterpolator()
}

// Animaciones scaleX y scaleY para cada punto con delays distintos
val anim1X = createScaleAnimator(dot1, 0)
val anim2X = createScaleAnimator(dot2, 200)
val anim3X = createScaleAnimator(dot3, 400)

val anim1Y = ObjectAnimator.ofFloat(dot1, "scaleY", 1f, 1.6f, 1f).apply {
    duration = 600
    startDelay = 0
    repeatCount = ObjectAnimator.INFINITE
    repeatMode = ObjectAnimator.RESTART
    interpolator = AccelerateDecelerateInterpolator()
}
val anim2Y = ObjectAnimator.ofFloat(dot2, "scaleY", 1f, 1.6f, 1f).apply {
    duration = 600
    startDelay = 200
    repeatCount = ObjectAnimator.INFINITE
    repeatMode = ObjectAnimator.RESTART
    interpolator = AccelerateDecelerateInterpolator()
```

```
        }
        val anim3Y = ObjectAnimator.ofFloat(dot3, "scaleY", 1f, 1.6f, 1f).apply {
            duration = 600
            startDelay = 400
            repeatCount = ObjectAnimator.INFINITE
            repeatMode = ObjectAnimator.RESTART
            interpolator = AccelerateDecelerateInterpolator()
        }

        // Reproducimos juntos todas las animaciones de los puntitos
        val dotsSet = AnimatorSet()
        dotsSet.playTogether(anim1X, anim2X, anim3X, anim1Y, anim2Y, anim3Y)
        dotsSet.start()

        // Temporizador para abrir LoginActivity después de 5 segundos
        Handler(Looper.getMainLooper()).postDelayed({
            startActivity(Intent(this@MainActivity, LoginActivity::class.java))
            finish()
        }, 5000)
    }
}
```

1.- Descripción del Código Fuente

El archivo MainActivity.kt corresponde a la **actividad principal de entrada** de la aplicación móvil **PickUp**, desarrollada en Android Studio con el lenguaje de programación **Kotlin**. Esta clase forma parte fundamental de la experiencia inicial del usuario, ya que combina la **inicialización de servicios esenciales**, una **animación de bienvenida atractiva** y una **transición automática hacia la interfaz de inicio de sesión**. A continuación, se describen de manera detallada los elementos y procesos involucrados en su funcionamiento.

2.- Inicialización de servicios

En la función onCreate, que se ejecuta al momento de lanzar la actividad, se lleva a cabo la inicialización del entorno de Firebase mediante la instrucción:

FirebaseApp.initializeApp(this)

Esta línea permite conectar la aplicación con los servicios en la nube de Firebase, como autenticación, base de datos y almacenamiento. Su activación desde la pantalla inicial asegura que cualquier funcionalidad posterior que dependa de Firebase esté disponible desde el principio.

3.- Carga de la vista principal

Se define el diseño de la interfaz a utilizar mediante la línea:

setContentView(R.layout.activity_main)

Esto vincula el código con el archivo XML correspondiente a la interfaz visual (activity_main.xml), que contiene los elementos gráficos que serán animados, como el logotipo, texto de bienvenida y puntos animados.

4.- Animaciones de bienvenida

Una de las características más destacables de esta actividad es la **implementación de animaciones para ofrecer una experiencia visual más fluida y moderna** al usuario desde el primer momento.

5.- Animación del logotipo

Se aplica un conjunto de animaciones al logotipo (logoImageView), específicamente:

- **Transparencia (alpha)**: el logotipo aparece gradualmente (de 0 a 100%).
- **Traslación vertical (translationY)**: el logotipo se desplaza suavemente desde una posición más baja hacia su ubicación final.

Estas animaciones son agrupadas en un objeto AnimatorSet, ejecutadas con una duración de 1 segundo y un interpolador AccelerateDecelerateInterpolator para lograr una aceleración natural.

6.- Animación del texto de bienvenida

Un efecto similar se aplica al texto de bienvenida (bienvenidoText), con una ligera demora de 500 milisegundos para que aparezca después del logotipo, logrando así una secuencia armoniosa de entrada.

7.- Animación de puntos indicadores

Además del logotipo y el texto, se incorporan **tres elementos circulares animados** (dot1, dot2, dot3) que simulan un patrón de carga mientras el usuario espera.

Funcionamiento:

- Cada punto se anima con un cambio de tamaño (escala en los ejes X y Y) de forma cíclica, es decir, se expande y contrae en bucle.
- Las animaciones están programadas con diferentes tiempos de retardo (0 ms, 200 ms, 400 ms) para cada punto, generando un efecto visual dinámico y escalonado que sugiere actividad de carga.
- Todas las animaciones de los puntos son agrupadas en un AnimatorSet llamado dotsSet, que se ejecuta de forma simultánea.

Este recurso visual aporta una **sensación de interactividad y fluidez**, mejorando notablemente la percepción del usuario en los primeros segundos de uso de la app.

Transición automática hacia LoginActivity

Luego de presentar las animaciones iniciales, se programa un **temporizador de 5 segundos** utilizando la clase Handler junto con postDelayed. La función programada ejecuta el siguiente bloque:

```
startActivity(Intent(this@MainActivity, LoginActivity::class.java))

finish()
```

Con ello, se lanza la actividad de inicio de sesión (LoginActivity) y se cierra automáticamente la MainActivity para que el usuario no pueda regresar con el botón de retroceso, manteniendo así una navegación lógica y controlada dentro de la aplicación.

Resumen de responsabilidades

Elemento	Función específica
FirebaseApp.initializeApp()	Inicializa los servicios de Firebase desde el arranque de la aplicación.
setContentView()	Establece la vista principal de la pantalla inicial.
AnimatorSet (logotipo y texto)	Crea animaciones de entrada con fade in y desplazamiento vertical.
AnimatorSet (puntos)	Ejecuta animaciones de escala secuencial para mostrar una carga animada.
Handler().postDelayed()	Controla el tiempo de espera antes de redirigir al usuario al inicio de sesión.
startActivity() + finish()	Cambia de pantalla y evita navegación inversa hacia la animación de inicio.

Importancia del archivo MainActivity.kt en la arquitectura de la aplicación

Esta clase actúa como una **pantalla de presentación** (Splash Screen extendido con animación personalizada), cuya misión principal es garantizar que el usuario comience la experiencia de forma atractiva y que, al mismo tiempo, se prepare silenciosamente la estructura básica del sistema (autenticación y navegación). Aunque no contiene lógica compleja de negocio, **su rol es crucial para la percepción profesional y funcional de la aplicación**, especialmente en proyectos orientados a usuarios finales.

11. Flujo de Datos

10.1 Firebase Firestore

Colecciones principales:

- users/ → Información de usuarios
- comidas/ → Productos disponibles
- historial/ → Pedidos realizados
- payment_methods/ → Métodos de pago guardados

10.2 Room Database (Offline First)

Tablas:

- LocalOrder → Pedidos locales antes de sincronizar
- LocalOrderProduct → Productos asociados a un pedido
- LocalPaymentMethod → Métodos de pago guardados localmente
- LocalProduct → Cache de productos para modo offline

10.3 Sincronización con WorkManager

SyncWorker:

- Verifica cambios locales y remotos
- Sube datos pendientes a Firestore
- Descarga actualizaciones cuando hay conexión
- Se ejecuta cada 1 hora o cuando se detecta conectividad

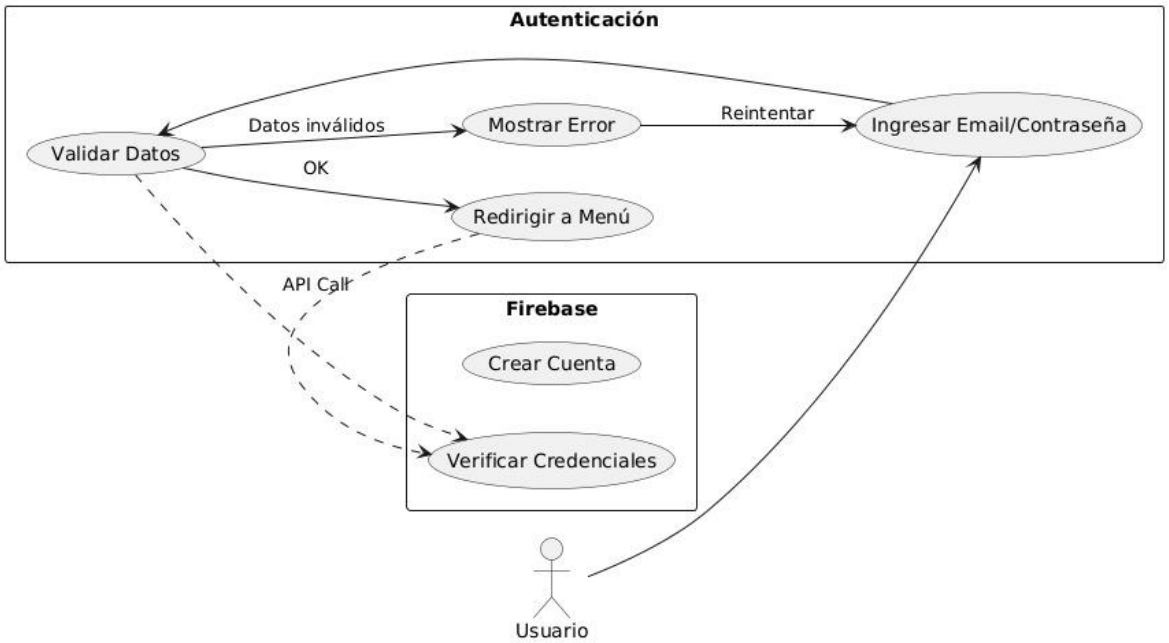
```
class SyncWorker(context: Context, params: WorkerParameters) : CoroutineWorker(context, params) {
    override suspend fun doWork(): Result {
        return try {
            syncLocalData()
            downloadRemoteData()
            Result.success()
        } catch (e: Exception) {
            Result.retry()
        }
```

```
}
```

12. Análisis de Casos de Uso

1.- Gestión de Autenticación y Acceso

Este diagrama muestra los casos de uso relacionados con el acceso al sistema:



- **Registrarse**: Permite a nuevos usuarios crear una cuenta en la aplicación ingresando un email y contraseña
- **Iniciar Sesión**: Autenticación de usuarios existentes
- **Gestionar Perfil**: Actualización de información personal del usuario
- **Cerrar Sesión**: Finalización segura de la sesión

Precondiciones y Postcondiciones:

- **Precondiciones**: El usuario debe tener acceso a internet y la aplicación instalada
- **Postcondiciones**: Usuario autenticado y redirigido al menú principal

Flujos alternativos más detallados:

- **Recuperación de contraseña**: Qué pasa si el usuario olvida su contraseña
- **Validación de email**: Proceso de verificación de correo electrónico

Requisitos técnicos:

- **Validaciones**: Formato de email, fortaleza de contraseña
- **Seguridad**: Encriptación de datos, protección de credenciales

2.- Gestión del Menú Principal y Gestión de Historial y Seguimiento

Este diagrama muestra un caso de uso general del sistema con los siguientes elementos:

Actor Principal:

- **Cliente**: Usuario que interactúa con la aplicación móvil

Sistemas Externos:

- **Firestore DB**: Base de datos para almacenamiento de información
- **Firebase Auth**: Sistema de autenticación

Casos de Uso Principales:

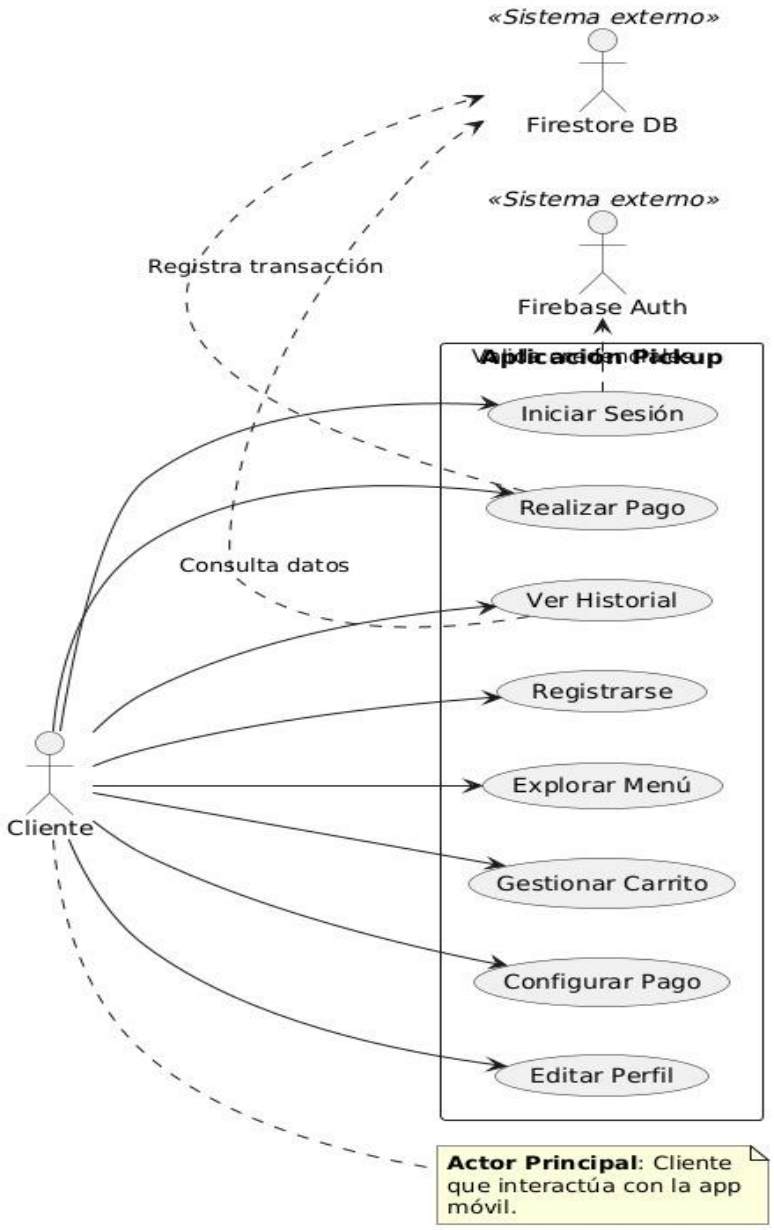
1. **Iniciar Sesión**: Autenticación del usuario en el sistema
2. **Realizar Pago**: Procesamiento de pagos por servicios
3. **Ver Historial**: Consulta de transacciones y servicios anteriores
4. **Registrarse**: Creación de nueva cuenta de usuario
5. **Explorar Menú**: Navegación por las opciones disponibles
6. **Gestionar Carrito**: Administración de servicios seleccionados
7. **Configurar Pago**: Configuración de métodos de pago
8. **Editar Perfil**: Modificación de información personal

Interacciones del Sistema:

- **Registra transacción**: El sistema guarda las operaciones realizadas
- **Consulta datos**: Recuperación de información almacenada
- Las líneas punteadas indican comunicación con sistemas externos (Firebase Auth y Firestore DB)

Características del Diagrama:

- Muestra la **vista completa del sistema** desde la perspectiva del cliente
- Integra servicios de **Firebase** como backend
- Cubre el **ciclo completo** de uso de la aplicación: desde registro hasta pago y seguimiento



3.- Gestión del Carrito de Servicios

Este diagrama de caso de uso muestra la **funcionalidad del carrito de compras** con los siguientes elementos:

Actor Principal:

- **Usuario**: Cliente que gestiona su carrito de compras

Sistema Externo:

- **Room DB**: Base de datos local para persistencia de datos

Módulos del Sistema:

Módulo Sistema:

- **Persistir Temporalmente**: Guarda datos de forma temporal
- **Recuperar Datos**: Obtiene información almacenada previamente

Módulo Carrito:

- **Agregar Producto**: Añade items al carrito
- **Ver Total**: Muestra el costo total de los productos
- **Eliminar Producto**: Remueve items del carrito

Casos de Uso Extendidos:

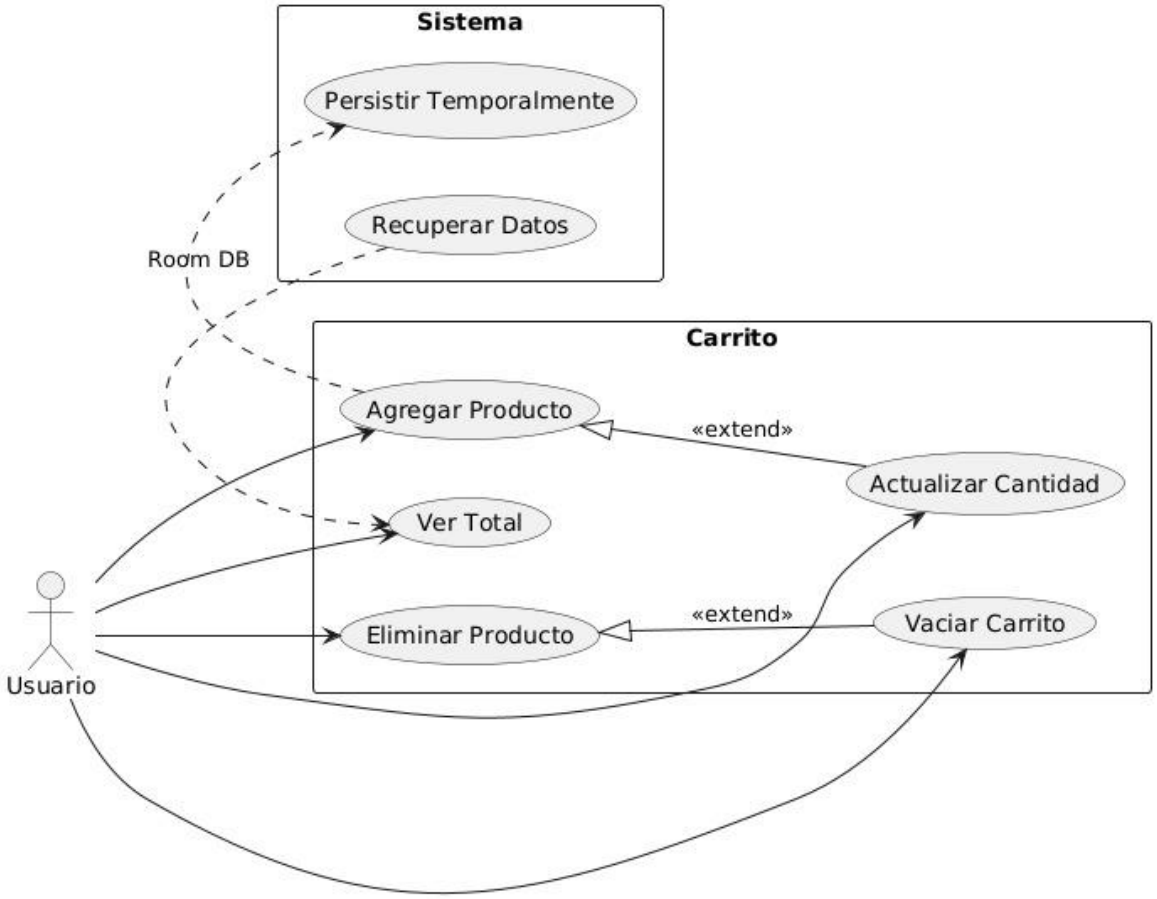
- **Actualizar Cantidad**: Extiende "Agregar Producto" - permite modificar cantidades
- **Vaciar Carrito**: Extiende "Eliminar Producto" - limpia completamente el carrito

Características Técnicas:

- **Persistencia Local**: Utiliza Room DB para almacenamiento offline
- **Relaciones de Extensión**: Los casos "Actualizar Cantidad" y "Vaciar Carrito" extienden funcionalidades base
- **Modularidad**: Separación clara entre lógica del sistema y funcionalidades del carrito

Flujo Principal:

1. Usuario añade productos al carrito
2. Sistema persiste temporalmente la información
3. Usuario puede ver el total, actualizar cantidades o eliminar productos
4. Los datos se recuperan según sea necesario desde Room DB



4.- Procesamiento de Pagos

Casos de uso relacionados con transacciones financieras:

Actor Principal:

- **Usuario**: Cliente que realiza el pago

Sistema Principal - Pago:

Contiene todos los casos de uso relacionados con el procesamiento de pagos

Casos de Uso Principales:

Selección de Método de Pago:

- **Seleccionar Método**: El usuario elige entre las opciones disponibles
- **Reintentar**: Permite volver a seleccionar si hay problemas

Métodos de Pago Disponibles:

1. **Tarjeta**:
    - **Introducir Tarjeta**: Ingreso de datos de tarjeta de crédito/débito
    - **Confirmar Pago**: Validación y procesamiento del pago con tarjeta
2. **PayPal**:
    - **Introducir PayPal**: Acceso a la cuenta de PayPal del usuario

Flujos de Procesamiento:

- **Confirmar Pago** → **OK** → **Mostrar Éxito** (pago exitoso)
- **Confirmar Pago** → **Falló** → **Mostrar Error** (pago fallido)

Sistemas Externos - Pasarelas de Pago:

- **PayPal API**: Procesamiento de pagos a través de PayPal
- **Stripe API**: Procesamiento de pagos con tarjeta a través de Stripe

Integración Técnica:

- **OAuth**: Autenticación segura para PayPal
- **Tokenización**: Proceso de seguridad para el manejo de datos sensibles

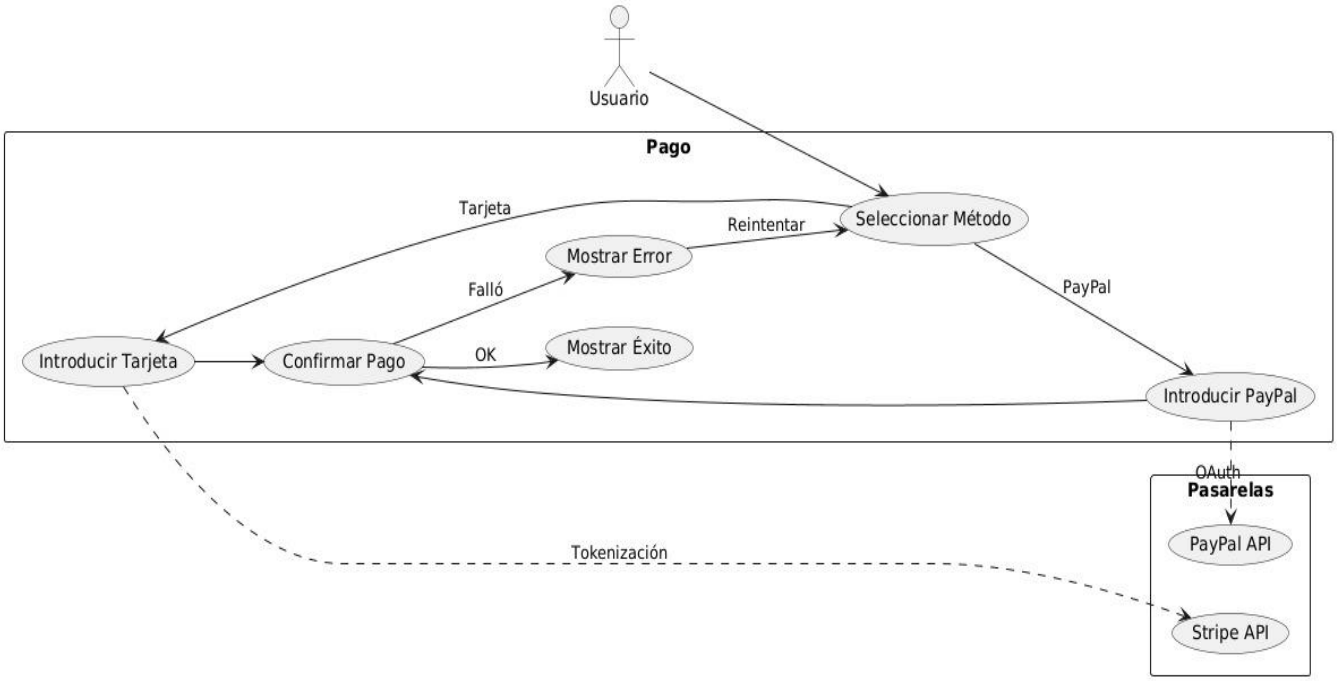
- **APIs Externas**: Comunicación con servicios de pago terceros

Características del Sistema:

- **Múltiples métodos**: Soporte para tarjeta y PayPal
- **Manejo de errores**: Flujos alternativos en caso de fallo
- **Seguridad**: Uso de APIs certificadas y tokenización
- **Experiencia de usuario**: Opciones de reintento y mensajes claros de éxito/error

Flujo Principal:

1. Usuario selecciona método de pago
2. Introduce credenciales (tarjeta o PayPal)
3. Sistema confirma el pago a través de la API correspondiente
4. Muestra resultado (éxito o error) al usuario



5.- Caso de uso - Flujo Principal del Sistema

Este diagrama muestra el flujo completo de navegación de la aplicación con los siguientes elementos:

Actividades Principales:

- **MainActivity**: Punto de entrada de la aplicación
- **LoginActivity**: Pantalla de autenticación
- **MenuActivity**: Menú principal de la aplicación
- **CarritoActivity**: Gestión del carrito de compras
- **Historial**: Consulta de pedidos anteriores
- **Perfil**: Gestión de información personal
- **PaymentSettingsActivity**: Configuración de métodos de pago

Flujos de Navegación:

Flujo de Autenticación:

1. **MainActivity** → **LoginActivity** (si no autenticado)
2. **LoginActivity** → **Registrarse** (clic en registrarse)
3. **LoginActivity** → **MenuActivity** (login exitoso)
4. **MainActivity** → **MenuActivity** (si ya logueado)

Flujo del Menú Principal:

- **MenuActivity** es el centro de navegación que conecta con:
    - **CarritoActivity** (ir al carrito)
    - **Historial** (ver historial)
    - **Perfil** (ver perfil)
    - **PaymentSettingsActivity** (configurar pago)

Flujo de Compras:

1. **MenuActivity** → **CarritoActivity** (confirmar compra)
2. **CarritoActivity** → **MenuActivity** (seguir comprando)
3. **MenuActivity** → **PaymentSettingsActivity** (si no tiene método guardado)

Flujo de Pago:

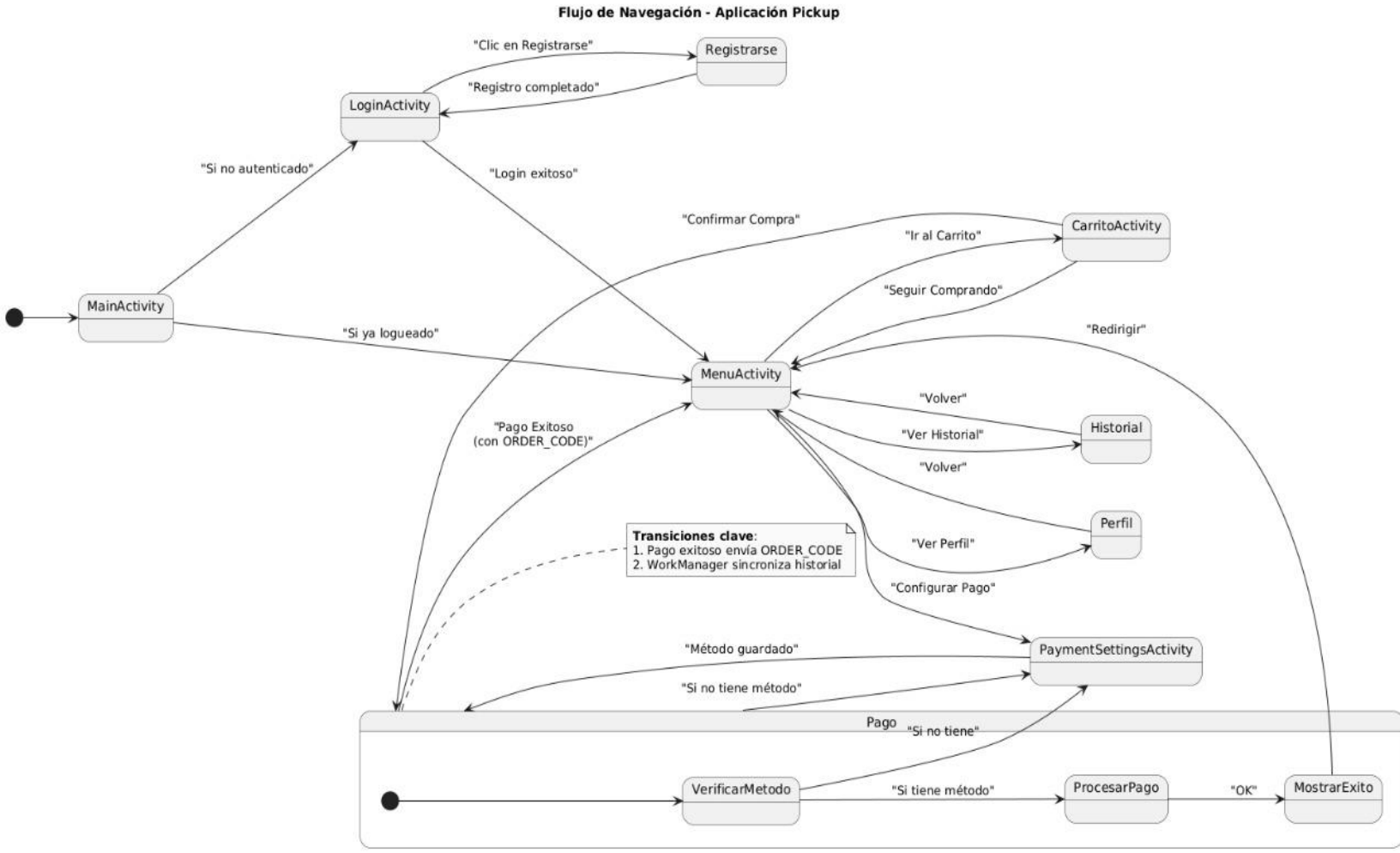
- **Módulo de Pago** independiente con:
    - **VerificarMetodo** → **ProcesarPago** (si tiene método)
    - **ProcesarPago** → **MostrarExito** (pago OK)

Transiciones Clave:

- **Navegación circular**: Permite volver desde cualquier pantalla
- **Validaciones**: Verifica autenticación y métodos de pago
- **Estados condicionales**: Diferentes rutas según el estado del usuario
- **Integración de pagos**: Flujo separado para procesamiento de transacciones

Características Técnicas:

- **WorkManager**: Sincroniza historial en segundo plano
- **ORDER_CODE**: Manejo de códigos de pedido para transacciones
- **Persistencia**: Guarda métodos de pago y datos de sesión



13. Diseño de la Aplicación

Principios de Diseño:

- **Material Design 3:** Componentes modernos y consistentes
- **Diseño minimalista:** Navegación clara y simple
- **Responsive:** Adaptación a diferentes tamaños de pantalla
- **Accesibilidad:** Contraste adecuado y navegación por teclado

Paleta de Colores:

- **Primario:** #6200EE (Purple)
- **Secundario:** #03DAC6 (Teal)
- **Superficie:** #FFFFFF (White)
- **Error:** #B00020 (Red)

Componentes UI:

- Cards para productos
- FAB para acciones principales
- Bottom Navigation para navegación
- Snackbars para feedback
- Dialogs para confirmaciones

14. Validaciones y Manejo de Errores

Validaciones Implementadas:

- **Email:** Formato válido y no vacío
- **Contraseña:** Mínimo 6 caracteres, al menos una mayúscula y un número
- **Tarjeta de crédito:** Validación de Luhn y formato
- **Campos obligatorios:** Verificación de completitud
- **Conexión de red:** Verificación antes de operaciones críticas

Manejo de Errores:

- **Errores de red:** Reintentos automáticos con backoff exponencial
- **Tokens inválidos:** Renovación automática de sesión
- **Errores de base de datos:** Fallback a datos locales
- **Errores de UI:** Feedback visual inmediato al usuario

```
sealed class NetworkResult<T> {
    data class Success<T>(val data: T) : NetworkResult<T>()
    data class Error<T>(val message: String) : NetworkResult<T>()
    data class Loading<T>(val isLoading: Boolean = true) : NetworkResult<T>()
}
```

15. Seguridad

Medidas de Seguridad Implementadas:

- **HTTPS:** Todas las comunicaciones encriptadas
- **Tokens seguros:** JWT con expiración automática
- **Sin almacenamiento local de datos sensibles:** Datos críticos solo en memoria
- **Validación del lado del servidor:** Verificación de permisos en Firebase
- **Autenticación multifactor:** Opcional para mayor seguridad

Reglas de Firestore:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if request.auth != null;
    }
  }
}
```

16. Pruebas y Validación

Tipos de Pruebas:

- **Pruebas unitarias:** Lógica de negocio y utilidades
- **Pruebas de integración:** Interacción entre componentes
- **Pruebas de UI:** Automatizadas con Espresso
- **Pruebas de usabilidad:** Testing con usuarios reales
- **Pruebas de rendimiento:** Memoria y CPU
- **Pruebas en dispositivos reales:** Diferentes versiones de Android

Herramientas de Testing:

- JUnit 4 para pruebas unitarias
- Mockito para mocking
- Espresso para pruebas de UI
- Firebase Test Lab para pruebas en la nube

Pruebas de Conexión:

```
@Test
fun testSyncWorker() {
    val request = SyncWorker.buildRequest()
    WorkManager.getInstance(context).enqueue(request)
    // Verificar sincronización
}
```

17. Conclusión

La aplicación Pick Up fue concebida como una solución moderna a los problemas de eficiencia y comodidad en el proceso de pedidos de comida rápida. El enfoque multidisciplinario y colaborativo permitió no solo desarrollar una aplicación funcional, sino también consolidar conocimientos de desarrollo móvil, arquitectura de software y experiencia de usuario.

La implementación de Firebase simplificó la conectividad y permitió una escalabilidad sólida, mientras que la arquitectura MVVM con enfoque offline-first garantiza una experiencia de usuario fluida incluso sin conexión. El uso de tecnologías modernas como Kotlin, Room Database y WorkManager asegura un código mantenible y eficiente.

Se espera que este proyecto evolucione y pueda integrarse a negocios reales que busquen digitalizar sus servicios de atención al cliente, proporcionando una base sólida para futuras mejoras y expansiones.

18. Fuentes

1. Statista. (2023). *Share of online food delivery services in Mexico in 2023, by platform*. https://www.statista.com/statistics/1286250/mexico-market-share-food-delivery-platforms/
2. Deloitte. (2022). *Consumer trends in the food service industry*. https://www2.deloitte.com/uk/en/pages/consumer-industrial-products/articles/food-service-trends.html
3. McKinsey & Company. (2022). *The state of the restaurant industry: Challenges and opportunities*. https://www.mckinsey.com/industries/retail/our-insights/the-state-of-the-restaurant-industry
4. National Restaurant Association. (2023). *State of the Restaurant Industry Report 2023*. https://restaurant.org/research-and-media/research/research-reports/state-of-the-restaurant-industry-report
5. Forbes Business Council. (2022). *Why mobile ordering is critical for fast food success*. https://www.forbes.com/sites/forbesbusinesscouncil/2022/03/14/why-mobile-ordering-is-critical-for-fast-food-success/
6. Square. (2022). *Future of Restaurants Report: 2022 Edition*. https://squareup.com/us/en/the-bottom-line/reports/future-of-restaurants-2022
7. PYMNTS. (2022). *Digital ordering grows: How consumers use apps to streamline their food purchases*. https://www.pymnts.com/news/mobile-commerce/2022/digital-ordering-grows-consumers-use-apps-streamline-food-purchases/
8. Peterson, H. (2021). *Chipotle is growing sales by pushing customers to pick up their own food*. https://www.businessinsider.com/chipotle-pickup-lanes-digital-orders-drive-growth-2021-8
9. Kane, G. C., Palmer, D., Phillips, A. N., Kiron, D., & Buckley, N. (2020). *The digital transformation of the restaurant industry*. https://hbr.org/2020/10/the-digital-transformation-of-the-restaurant-industry
10. Constine, J. (2022). *How Domino's and Starbucks are dominating the digital ordering game*. https://techcrunch.com/2022/04/21/starbucks-dominos-digital-strategy-delivery/
11. Android Developers. (2024). *Android Architecture Components*. https://developer.android.com/topic/architecture
12. Firebase Documentation. (2024). *Firebase for Android*. https://firebase.google.com/docs/android/setup
13. Material Design. (2024). *Material Design 3*. https://m3.material.io/